

STM32F407 双步进电机蓝牙控制说明书

版本：2026-07-27 适用工程：F407_Emm_SingleTurnHold 电机地址：电机 1 为 1，电机 2 为 2

本说明书按当前工程源码编写。当前蓝牙控制协议是 ASCII 文本协议；发送端必须发送字符本身，而不是把角度作为二进制整数直接发出。

1. 最快使用方法

- 1 给 F407、蓝牙模块和两台电机正确供电，所有设备共地。
- 2 两台电机的 TTL 串口地址分别设为 1 和 2，波特率均为 115200。
- 3 上电后等待程序自动使能并回到两台电机各自保存的零点，约需 4 秒。
- 4 使用网页上位机时，打开 [WebUpperComputer/start_web.bat](#)，在 Edge 或 Chrome 中选择蓝牙模块并连接。
- 5 使用另一块 STM32 时，通过它的串口向发送端蓝牙模块写入以下 ASCII 字符：

```
[TEXT]
SET,300,600\r\n
```

这条命令表示电机 1 绝对转到 +30.0°，电机 2 绝对转到 +60.0°。其中 \r\n 必须变成实际的 0x0D 0x0A 两个字节，不能发送成四个可见字符 \、r、\、n。

2. 系统组成和串口分工

功能	F407 外设	引脚	参数	程序文件
蓝牙和外部控制	USART2	PD5/TX, PD6/RX	9600, 8N1, 无流控	Src/bluetooth_app.c
两台 Emm 电机总线	UART4	PC10/TX, PC11/RX	115200, 8N1, 无流控	Src/motor_uart.c 、 Src/Emm_V5.c
用户按键	GPIO	PE0, 低电平有效，上拉	10 ms 扫描	Src/key_app.c 、 Src/gpio.c
SSD1306 OLED	软件 SPI	PD11-PD14	GPIO 模拟时序	Src/oled.c 、 Src/oled_app.c

主循环不阻塞地反复调用：

```
[C]
key_pro();
uart_pro();
bluetooth_pro();
motor_pro();
oled_pro();
```

蓝牙 USART2 使用单字节中断接收。UART4 使用 Receive-to-IDLE DMA 接收电机位置帧；HAL 回调只保存数据和重启接收，帧解析放在主循环中执行。

3. 接线

3.1 接收端 F407 与蓝牙模块

F407	接收端蓝牙模块
PD5 / USART2_TX	RXD
PD6 / USART2_RX	TXD
GND	GND

TX 和 RX 必须交叉。蓝牙模块 TXD 送入 PD6 的逻辑高电平必须与 3.3 V MCU 兼容；不要把 5 V TTL 直接送入 PD6。

3.2 F407 与两台电机

F407	两台电机共用的 TTL 总线
PC10 / UART4_TX	电机 1 TTL_RX 和电机 2 TTL_RX
PC11 / UART4_RX	电机 1 TTL_TX 和电机 2 TTL_TX
GND	两台电机 GND

两台电机必须分别设置地址 1、2，并关闭主动周期上报，避免两台电机同时占用 RX 线。电机动力电源单独供电，但必须与 F407 共地。

3.3 另一块 STM32 与发送端蓝牙模块

发送端 STM32	发送端蓝牙模块
UART_TX	RXD
UART_RX	TXD，可选但建议连接，以便读取应答
GND	GND

两只蓝牙模块必须已经建立透明串口连接。接收端模块是从机/外设时，发送端模块需要具备主机/中心设备能力并完成配对或连接。仅仅给两个模块设置相同名称，不会自动建立通信。

如果不经蓝牙而直接用导线测试，可将发送端 STM32 的 TX 接目标 F407 的 PD6，发送端 RX 接目标 F407 的 PD5，并共地。

4. 通信规则

项目	规定
编码	ASCII
波特率	9600 bit/s
数据格式	8 数据位，1 停止位，无校验，即 8N1
命令结束	推荐 CR LF，即 0x0D 0x0A；程序实际以 LF 判定结束
大小写	命令关键字必须大写：SET、STAT?
空格	推荐完全不加空格
角度单位	命令和状态中的整数均以 0.1° 为单位
角度范围	-3600 到 3600，即 -360.0° 到 +360.0°
正负方向	正数为顺时针，负数为逆时针
C 字符串结尾	不发送 \0；只发送有效字符和 \r\n

例如 -45.0° 应写成整数 -450。正数推荐写 300，不需要写 +300。

5. 完整指令表

当前固件只实现了 SET 和 STAT? 两条外部输入命令。

指令	实际发送格式	功能	成功回复
设置双电机绝对目标	SET,<M1>,<M2>\r\n	同时设置电机 1、2 的绝对角度	OK,SET,<M1>,<M2>\r\n

指令	实际发送格式	功能	成功回复
查询状态	STAT?\r\n	立即返回一条完整状态	STAT,...\r\n

错误关键字、缺少参数、多余字符、未发送换行或角度越界均不能作为有效运动命令。能够形成一条完整但不合法的命令时，固件回复：

```
[TEXT]
ERR,BAD_CMD\r\n
```

5.1 常用运动指令

目标动作	发送内容
两台回到 0°	SET,0,0\r\n
两台绝对转到 +30.0°	SET,300,300\r\n
两台绝对转到 +60.0°	SET,600,600\r\n
两台绝对转到 +90.0°	SET,900,900\r\n
两台绝对转到 -30.0°	SET,-300,-300\r\n
电机 1 到 +30.0°，电机 2 保持 0°	SET,300,0\r\n
电机 1 到 -45.0°，电机 2 到 +90.0°	SET,-450,900\r\n
立即查询状态	STAT?\r\n

SET 是绝对坐标命令。连续发送两次 SET,300,300，目标仍然是 30.0°，不会变成 60.0°。如果另一块 STM32 也要实现“每次增加 30°”，发送端应自己保存目标值并依次发送 SET,300,300、SET,600,600、SET,900,900。

6. 字节级示例

命令：

```
[TEXT]
SET,300,600\r\n
```

共 13 个字节：

字符	S	E	T	,	3	0	0	,	6	0	0	CR	LF
十六进制	53	45	54	2C	33	30	30	2C	36	30	30	0D	0A

下面两种错误发送都不能代替上述 13 个 ASCII 字节：

- 直接发送两个 16 位二进制整数 300、600。
- 发送可见字符串 SET,300,600\r\n，其中最后四字节是 5C 72 5C 6E。

7. 另一块 STM32 的 HAL 发送程序

以下例程假设发送端蓝牙模块连接到 huart1。如果你的工程使用其他串口，只需替换句柄。

7.1 发送固定角度

```
[C]
#include "usart.h"

void Bluetooth_Send_Test(void)
{
    static const uint8_t command[] = "SET,300,600\r\n";

    HAL_UART_Transmit(&huart1,
        (uint8_t *)command,
        sizeof(command) - 1U,
        100U);
}
```

`sizeof(command) - 1U` 会排除 C 字符串末尾的 `\0`，但字符串中的 `\r\n` 会由编译器变成实际的 `0x0D 0x0A`。

7.2 发送任意双电机角度

```
[C]
#include "usart.h"
#include <stdio.h>

uint8_t Motor_SendAbsoluteAngle(int16_t motor1_angle10,
    int16_t motor2_angle10)
{
    char command[32];
    int length;

    if ((motor1_angle10 < -3600) || (motor1_angle10 > 3600) ||
        (motor2_angle10 < -3600) || (motor2_angle10 > 3600))
    {
        return 0U;
    }

    length = sprintf(command, "SET,%d,%d\r\n",
        motor1_angle10, motor2_angle10);
    if (length <= 0)
    {
        return 0U;
    }

    if (HAL_UART_Transmit(&huart1, (uint8_t *)command,
        (uint16_t)length, 100U) != HAL_OK)
    {
        return 0U;
    }

    return 1U;
}
```

调用示例：

```
[C]
Motor_SendAbsoluteAngle(300, 600); /* +30.0°, +60.0° */
Motor_SendAbsoluteAngle(-450, 900); /* -45.0°, +90.0° */
Motor_SendAbsoluteAngle(0, 0); /* 两台回到零点 */
```

7.3 发送端实现“每按一次增加 30°”

```
[C]
static int16_t target1_angle10 = 0;
static int16_t target2_angle10 = 0;

void Sender_KeyPressed(void)
{
    if ((target1_angle10 <= 3300) && (target2_angle10 <= 3300))
    {
        target1_angle10 += 300;
        target2_angle10 += 300;
        Motor_SendAbsoluteAngle(target1_angle10, target2_angle10);
    }
}
```

这是在发送端把绝对目标累加。目标 F407 上 PE0 按键的内部实现则不同，它直接向两台电机各装载约 30° 的相对运动。

7.4 查询状态

```
[C]
void Motor_QueryStatus(void)
{
    static const uint8_t command[] = "STAT?\r\n";

    HAL_UART_Transmit(&huart1,
        (uint8_t *)command,
        sizeof(command) - 1U,
        100U);
}
```

建议发送端也启用 UART 接收，以换行 `\n` 为一帧结束，读取 **OK**、**ERR** 和 **STAT**。不要连续高速发送多条命令；最稳妥的做法是发送一条后等待 **OK,SET**，并通过 **STAT** 中的状态和角度判断运动是否完成。

8. 回复和状态数据

8.1 启动提示

蓝牙接收初始化后，F407 发送：

```
[TEXT]
BT READY\r\n
```

8.2 设置命令应答

发送：

```
[TEXT]
SET,300,600\r\n
```

解析成功后回复：

```
[TEXT]
OK,SET,300,600\r\n
```

OK,SET 表示格式、范围检查通过并已保存待执行目标，不代表电机已经到位。到位应以 **STAT** 中的实际角度接近目标角度且 **state=5** 为准。

8.3 完整状态格式

固件每秒主动发送一次状态，收到 **STAT?** 时也会立即发送一次：

```
[TEXT]
STAT,actual1,actual2,target1,target2,state,bt_count,last_byte,bt_error,rx_state,uart_sr\r\n
```

示例：

```
[TEXT]
STAT,300,601,300,600,5,13,10,0,34,192\r\n
```

序号	字段	含义
1	actual1	电机 1 实际角度，单位 0.1°
2	actual2	电机 2 实际角度，单位 0.1°
3	target1	电机 1 当前保存目标，单位 0.1°
4	target2	电机 2 当前保存目标，单位 0.1°
5	state	电机控制状态机编号
6	bt_count	USART2 累计收到的字节数
7	last_byte	最后收到的字节，十进制；正常命令末尾 LF 为 10
8	bt_error	蓝牙接收、UART 或命令解析错误累计值
9	rx_state	HAL 的 USART2 接收状态；正常等待接收常见为 34，即 0x22
10	uart_sr	USART2 状态寄存器十进制值；正常空闲常见为 192，即 0x00C0

示例中 **actual2=601** 表示 60.1°，并不是 601°。

9. 状态机编号

状态	名称	程序正在做什么
0	上电等待	等待 500 ms，然后使能电机 1
1	使能电机 2	与上一条命令间隔 100 ms 后使能电机 2
2	电机 1 回零	向地址 1 发送回已保存零点命令
3	电机 2 回零	向地址 2 发送回已保存零点命令
4	等待回零	固定等待 3 s
5	保持	接收新目标、按键请求并检测外力偏移
6	按键装载电机 2	电机 1 的相对 +30° 已装载，等待装载电机 2
7	按键同步启动	等待向广播地址 0 发送同步启动
8	运动中	轮询两台位置，连续两组均到位后返回状态 5
9	回正电机 2	电机 1 绝对回正目标已装载，等待装载电机 2
10	回正同步	等待广播同步启动
11	回正中	轮询位置，到位后返回状态 5
12	外部设置电机 2	电机 1 绝对目标已装载，等待装载电机 2
13	外部设置同步	等待广播同步启动

正常路径：

[TEXT]

上电：0 -> 1 -> 2 -> 3 -> 4 -> 5

按键：5 -> 6 -> 7 -> 8 -> 5

SET：5 -> 12 -> 13 -> 8 -> 5

回正：5 -> 9 -> 10 -> 11 -> 5

10. 三种控制过程

10.1 上电自动回零

上电 500 ms 后，程序依次使能地址 1、2，再依次发送单圈就近回零命令。程序固定等待 3 秒后进入保持状态。正常运行时 `CALIBRATE_MOTOR_1_ON_BOOT` 和 `CALIBRATE_MOTOR_2_ON_BOOT` 必须都保持为 0。

当前程序按时间等待回零，没有读取“回零完成”标志。因此机械行程较长、速度较低或负载较大时，应确认 3 秒足够回零。

10.2 板载按键累计转动

PE0 配置为上拉输入，按下为低电平。程序每 10 ms 扫描一次，连续两次读到低电平才确认一次按下，并在松开前不重复触发。

一次有效按下会排入一个 $+30^\circ$ 请求。状态机先给电机 1、2 各装载 267 脉冲相对命令，再用地址 0 同步启动。因此连续按下后的累计目标约为 30° 、 60° 、 90° ，不是每次回到绝对 30° 。

10.3 外力偏移自动回正

保持、运动和回正阶段，程序每隔约 150 ms 轮流查询地址 1、2 的位置。只有两台电机都得到一组新数据后才比较实际位置和保存目标。

在保持状态下，若连续 3 组双电机采样中任意一台偏差超过 1.5° ，程序会给两台电机重新装载各自的绝对目标并同步启动。回正不修改保存目标，因此下一次按键仍从原来的累计目标继续 $+30^\circ$ 。

11. OLED 显示含义

OLED 字段	含义
M1、M2	两台电机实际角度
V	是否已经收到该电机的有效位置帧，1 为有效
KEY	PE0 有效按下次数，显示值对 1000 取余
CMD	已同步启动的按键或 SET 运动次数，显示值对 1000 取余
T1、T2	两台电机当前保存的目标角度
ST	当前状态机编号
PEND	尚未处理的按键 $+30^\circ$ 请求数
RXE	电机 UART/DMA/位置查询异常累计值
COR	自动回正同步启动次数
M1 ADDR:1 M2:2	当前电机地址配置
BT	USART2 累计接收字节数，显示值对 1000 取余
LAST	USART2 最后收到字节的十六进制值；完整命令后应常见 0A

OLED 接线为：PD11/DC、PD12/RST、PD13/DATA、PD14/CLK、3.3 V 和 GND。当前驱动是 GPIO 软件 SPI，不是硬件 SPI，也不是 I2C。

12. 故障判断表

现象	先看什么	说明和处理
发送后 BT 不增加	PD6、共地、模块连接、9600 8N1	字节没有到达 USART2
BT 增加但 LAST 不是 0A	发送末尾	没有发送实际 LF，命令不会进入解析
收到 ERR,BAD_CMD	大小写、逗号、空格、范围	严格按 SET,300,600\r\n 或 STAT?\r\n 发送
收到 OK,SET, CMD 暂时不加	ST 和当前运动	OK 是已入队；状态机在保持状态时才装载新目标
ST 按 5 -> 12 -> 13 -> 8 变化	实际角度	SET 已进入电机控制流程
CMD 增加但角度不变	UART4、电机供电、地址	蓝牙协议已成功，应检查电机总线和驱动器
RXE 持续增加	PC11、地址冲突、主动上报	电机位置帧超时或 DMA 接收异常
COR 持续增加	机械负载、位置噪声、零点	实际位置持续超过 1.5° 容差
第二条命令覆盖第一条	发送节奏	不要高速连发；等待应答和状态完成后再发下一目标

接收行缓冲区总长为 48 字节。过长命令会丢弃并增加 bt_error。为了避免正在处理上一行时丢失新字节，发送端应一次只发一条短命令，并等待应答。

13. 零点校准

正常固件必须保持：

```
[C]
#define CALIBRATE_MOTOR_1_ON_BOOT 0
#define CALIBRATE_MOTOR_2_ON_BOOT 0
```

重新标定某台电机时：

- 1 断开危险负载或确保机构不会碰撞，把对应电机手动放到真实零点。
- 2 只把该电机对应宏临时改为 1。
- 3 编译、烧录并上电一次，让驱动器保存当前位置为零点。
- 4 立即把该宏恢复为 0。
- 5 重新编译和烧录正常固件，再执行上电回零测试。

不要让标定宏长期为 1，否则每次上电都会重新保存零点。

14. 关键参数和修改位置

参数集中在 `Src/motor_app.c` 顶部:

参数	当前值	含义
<code>MOTOR_1_ADDRES</code>	1	电机 1 地址
<code>MOTOR_2_ADDRES</code>	2	电机 2 地址
<code>MOTOR_PULSES_PER_REV</code>	3200	每圈脉冲数
<code>MOTOR_STEP_PULSES</code>	267	按键每次相对脉冲, 约 30.04°
<code>MOTOR_SPEED_RPM</code>	180	位置控制速度
<code>MOTOR_ACCELERATION</code>	40	位置控制加速度参数
<code>COMMAND_GAP_MS</code>	100 ms	两台电机装载命令间隔
<code>POSITION_POLL_MS</code>	150 ms	位置轮询间隔
<code>POSITION_TIMEOUT_MS</code>	600 ms	单次位置应答超时
<code>MOVE_TIMEOUT_MS</code>	10000 ms	运动/回正等待超时
<code>POSITION_TOLERANCE</code>	1.5°	到位和外力偏移判断容差

修改电机细分或每圈脉冲数后, 必须同步修改 `MOTOR_PULSES_PER_REV`, 否则 SET 角度、按键 30° 和实际角度换算都会不一致。

15. 程序文件职责

文件	职责
<code>Src/main.c</code>	外设初始化和五个周期处理函数的调度
<code>Src/bluetooth_app.c</code>	USART2 单字节中断接收、文本组帧、SET/STAT 解析和状态发送
<code>Src/motor_app.c</code>	双电机上电、回零、运动、位置保持和外力回正状态机
<code>Src/motor_uart.c</code>	UART4 Receive-to-IDLE DMA 接收和 S_CPOS 位置帧解析
<code>Src/key_app.c</code>	按键周期处理、计数和 +30° 请求
<code>Src/gpio.c</code>	PE0 三态消抖和 GPIO 初始化
<code>Src/oled_app.c</code>	OLED 诊断内容和 200 ms 刷新
<code>Src/oled.c</code> , <code>Src/font.c</code>	SSD1306 显存、字库和软件 SPI 底层
<code>Src/Emm_V5.c</code>	厂商 Emm V5 电机串口命令封装
<code>WebUpperComputer/</code>	可选择任意附近 BLE 设备的网页控制端, 设备需提供 FFE0 串口服务

16. 使用限制

- 当前 SET 协议只允许 -360.0° 到 $+360.0^{\circ}$ 的单圈绝对目标。
- 两台电机命令先后装载，再由广播命令同步启动；这是同步启动，不代表两台不同角度的运动一定同时结束。
- 固件每秒主动发送一条状态，发送端接收程序应能处理没有主动查询也会到来的 STAT 行。
- 蓝牙链路是透明串口。BLE GATT 的配对、连接和特征写入由蓝牙模块或上位机负责，F407 只处理模块从 USART2 输出的字节。
- 修改 CubeMX 后应检查 USART2、UART4、DMA、PD5/PD6、PC10/PC11、PE0 和 PD11-PD14 配置是否仍与本文一致。